



TP 3. Alineamientos de secuencias de a pares

Objetivos

1. Entender el funcionamiento básico del algoritmo de alineamiento de pares de secuencias de Needleman-Wunsch.
2. Aprender a interpretar un Dot-Plot, pudiendo identificar las regiones relevantes que contienen patrones.
3. Comprender los conceptos de identidad, similitud y homología de secuencias, y establecer una clara diferencia entre los mismos.

Alineamientos de secuencias de a pares

El alineamiento de secuencias de a pares comprende la asignación uno-a-uno de correspondencias entre los elementos que componen dichas secuencias sin alterar su orden. En dicho proceso tres eventos principales pueden tener lugar:

- **Match (M):** Cuando los elementos enfrentados son equivalentes.
- **Mismatch (m):** Cuando los elementos correspondientes son diferentes.
- **Gap (g):** Cuando un elemento de una secuencia no tiene par en la otra y se enfrenta a un espacio, caracterizado por un guión (-).
 - **Gap open:** Cuando se abre un gap.
 - **Gap extend:** Cuando se agregan gaps a continuación de otro gap.

Por ejemplo, si alineamos las secuencias AFGIVHKLIVS y AFGIHKIVS un posible resultado sería:

```
A F G I V H K L I V S
A F G I - H K - I V S
```

La principal función de los alineamientos es establecer una medida de **similitud**

entre las secuencias que participan en el mismo. Para ello es necesario definir un **sistema de puntuación** que pese cada uno de los eventos que tienen lugar en la construcción del alineamiento. Asimismo, este esquema de puntajes o *scoring* nos permitirá optimizar el alineamiento de forma tal que los algoritmos empleados elijan la correspondencia entre secuencias que maximice el puntaje o *score* global.

Existen varios algoritmos de alineamiento:

- Los **alineamientos globales** (o de Needleman-Wunsch por sus creadores), se realizan apareando todos los elementos de una secuencia con todos los elementos de la otra. Este tipo de alineamientos se utiliza principalmente para comparar dos secuencias que son similares en longitud.
- Los **alineamientos locales** (o de Smith-Waterman), parean únicamente parte de las secuencias y son útiles para identificar, por ejemplo, dominios en común.
- Los **alineamientos mixtos**, que combinan los dos anteriores.

Algoritmo de Needleman-Wunsch

El algoritmo de Needleman-Wunsch es un método de programación dinámica para el alineamiento global de secuencias, desarrollado por Saul Needleman y Christian Wunsch en 1970. Este algoritmo garantiza encontrar el alineamiento óptimo entre dos secuencias completas, desde el primer hasta el último carácter, maximizando la similitud global.

¿Cómo funciona?

El algoritmo se basa en tres pasos fundamentales:

1. **Inicialización:** Construye una matriz de puntuación donde las filas representan los caracteres de una secuencia y las columnas los de la otra. Los bordes se llenan con penalizaciones acumulativas por gaps (huecos).
2. **Llenado de la matriz:** Para cada celda, calcula el máximo entre tres valores posibles:
 - Coincidencia/sustitución: puntuación diagonal + match/

mismatch

- Inserción: puntuación de arriba + penalización por gap
- Deleción: puntuación de la izquierda + penalización por gap

3. **Trazado inverso (backtracking):** Desde la última celda (esquina inferior derecha), se reconstruye el alineamiento óptimo siguiendo el camino de las puntuaciones máximas hacia la esquina superior izquierda.

Dynamic programming

Dado un par de secuencias y un sistema de puntuación o *scoring* se pueden aplicar diversos algoritmos para encontrar el alineamiento que dé el mejor puntaje.

El algoritmo más popular utiliza un método matemático llamado **dynamic programming**. El mismo consiste en comparar ambas secuencias construyendo una matriz del alineamiento. Brevemente:

Pasos a seguir

1. Se comienza en el extremo superior izquierdo de la matriz, con un puntaje inicial de 0.
2. En cada paso, se calcula el costo que tiene aparejado desplazarse de una celda a la otra, dado el sistema de puntajes pre-establecido, y se elige la opción más favorable, es decir aquella que **maximice** el puntaje global del alineamiento.
3. En cada iteración se guarda el puntaje con el que se llegó a una celda dada y el movimiento que originó dicho camino o *path*, indicado típicamente con una flecha. Una vez que la matriz está completa en su totalidad se puede recorrer hacia atrás o realizar un *traceback*, desde el extremo inferior derecho al superior izquierdo, para reconstruir el alineamiento.

La principal ventaja de este método es que **siempre encuentra el alineamiento**

óptimo entre las secuencias dadas.

Sin embargo, una desventaja es que pueden existir **varios** alineamientos que satisfagan esta condición.

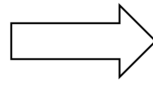
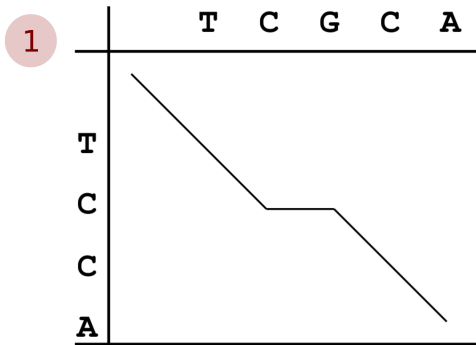
Otra desventaja es de origen técnica: la exhaustividad con la que el algoritmo realiza la búsqueda hace que su velocidad dependa de la longitud de las secuencias implicadas, haciendo poco eficiente la búsqueda de similitud de una secuencia contra una base de datos. Para esto existen diferentes adaptaciones del algoritmo que se verán más adelante.

Ejemplos

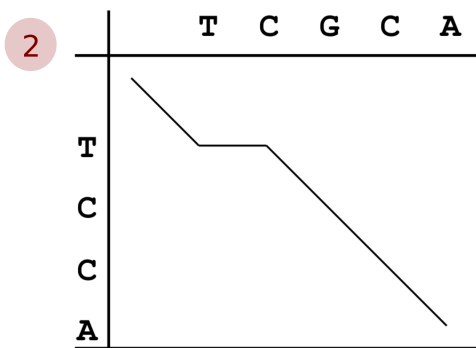
Imaginen que queremos alinear las secuencias **TCGCA** y **TCCA** utilizando un esquema de *scoring* de:

- **Match:** $M=1$
- **Mismatch:** $m=-1$
- **Gap:** $g=-2$

Para eso ubicamos las secuencias en una matriz, donde cada una de sus dimensiones corresponda a una de las secuencias, tal como se muestra en la siguiente figura.



$$\begin{array}{r}
 \text{TCGCA} \\
 \text{TC-CA} \\
 \hline
 1+1-2+1+1=2
 \end{array}
 \quad \text{Score} = 2$$



$$\begin{array}{r}
 \text{TCGCA} \\
 \text{T-CCA} \\
 \hline
 1-2-1+1+1=0
 \end{array}
 \quad \text{Score} = 0$$

Si observamos los *paths* **1** y **2** dibujados en las matrices de la figura podemos ver que se emplearon distintas estrategias para alinear este par de secuencias.

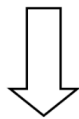
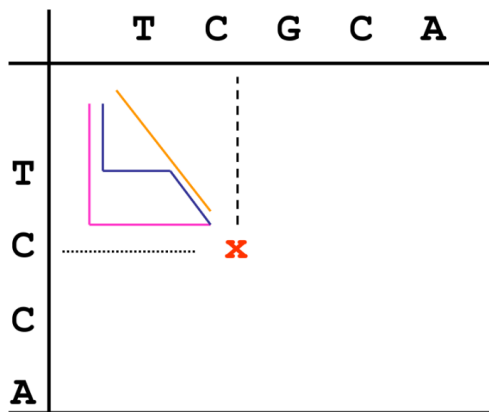
- En **1** se eligió alinear los dos primeros nucleótidos **TC** por la diagonal, luego colocar un **gap** en la secuencia vertical **TCCA** y para finalizar se alinearon los nucleótidos **CA** restantes por la diagonal.
- En **2** el primer nucleótido **T** de ambas secuencias se alineó por la diagonal, luego se colocó un **gap** en la secuencia vertical **TCCA** y finalmente se alinearon los 3 nucleótidos **GCA** y **CCA** restantes por la diagonal.

Si computamos los puntajes de ambos alineamientos, obtenemos que:

- La **opción 1** tiene un puntaje de 2. Se propone colocar un único **gap** permitiendo alinear al resto de los nucleótidos en ambas secuencias con eventos de **match**.

- La **opción 2** tiene un puntaje de 0. Las secuencias estudiadas se alinean con 1 **gap**, 1 **mismatch** y 3 **matches**. La estrategia es subóptima en relación a **1**.

Para comenzar, refresquemos cómo funcionaba el método de *dynamic programming*.



--TC
TC--

-TC
T-C

TC
TC

Para llegar desde el extremo superior izquierdo (= inicio) de la matriz del alineamiento a la posición marcada con una **x** podríamos, hipotéticamente, tomar cualquiera de los caminos dibujados en la figura de más arriba. Estos *paths* darían alineamientos diferentes de las secuencias **TC** con **TC**.

Para llegar a cualquier celda de la matriz, uno puede acceder por, como máximo, 3 direcciones. La idea es siempre moverse en la dirección que **maximice** el *score* o puntaje.

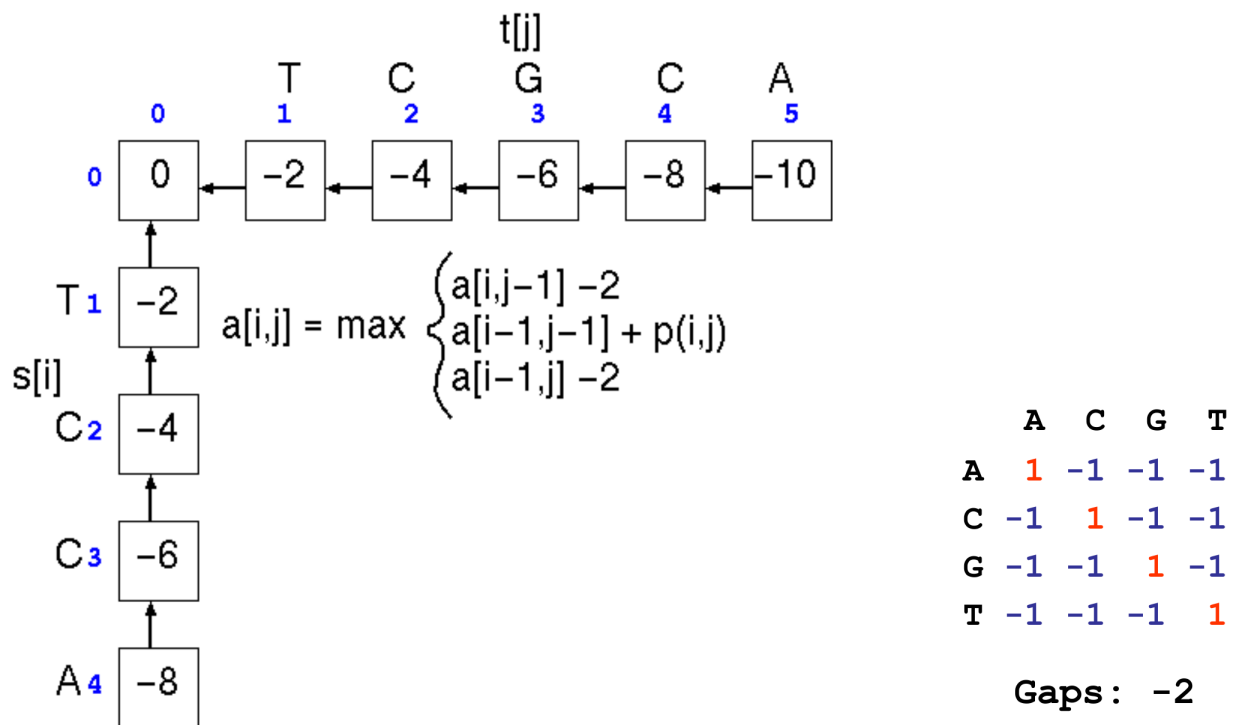
Veamos que:

- un movimiento en la dirección **horizontal**, de la posición **(i, j-1)** a la posición **(i, j)**, supone introducir un **gap** en la secuencia del eje vertical i

- un movimiento en la dirección **diagonal**, de la posición **(i-1, j-1)** a la posición **(i, j)**, supone un **match** o un **mismatch** entre los nucleótidos enfrentados
- un movimiento en la dirección **vertical**, de la posición **(i-1, j)** a la posición **(i, j)**, supone introducir un **gap** en la secuencia del eje horizontal j

Teniendo en cuenta la fórmula para obtener el *score* enunciada más arriba, podemos comenzar con nuestro ejercicio!

Recordemos que la matriz se llenará iterativamente, comenzando por la celda del extremo superior izquierdo, que tiene un puntaje de 0.



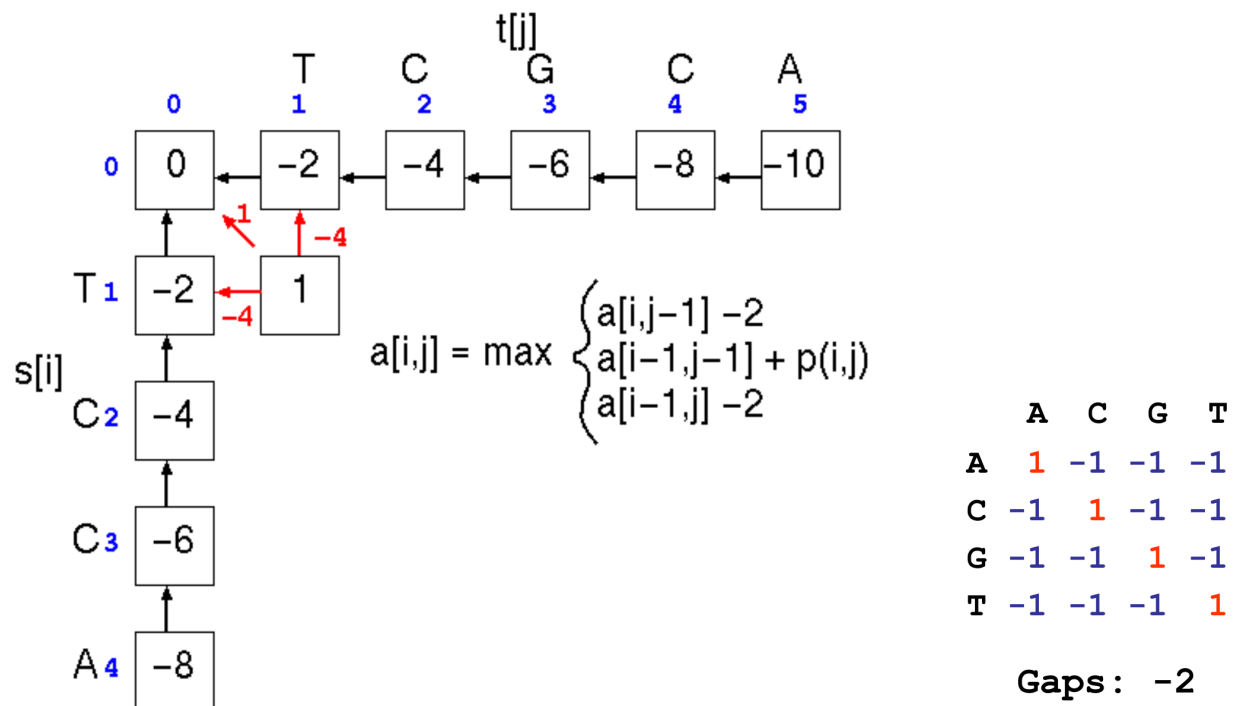
Para moverse del (0, 0) al (0, 1), hay una sólo opción, moverse en forma **horizontal**. Esto significa alinear **T** con un **gap**, lo cual da un score de $0 + (-2) = -2$.

eje j: T
eje i: -

Lo mismo pasa al moverse del (0, 0) al (1, 0), hay una sólo opción, moverse en forma **vertical**. Esto significa alinear **T** con un **gap**, lo cual también da un score de $0 + (-2) = -2$.

eje j: -
eje i: T

Para moverse del (0, 0) al (1, 1) hay 3 maneras:



1. Hacer un movimiento **vertical**, lo cual da un score de $-2 + (-2) = -4$

- **-2** es el puntaje de la celda inicial (0, 1)
- El movimiento vertical implica colocar un **gap**: -2

Resultado

eje j: T -
eje i: - T

2. Hacer un movimiento **horizontal**, lo cual da un score de $-2 + (-2) = -4$. Similar al caso anterior:

- **-2** es el puntaje de la celda inicial (1, 0)
- El movimiento horizontal implica colocar un **gap**: -2

Resultado

eje j: - T
eje i: T -

3. Hacer un movimiento **diagonal**, lo cual da un score de $0 + (+1)$. Implica alinear ambos nucleótidos!

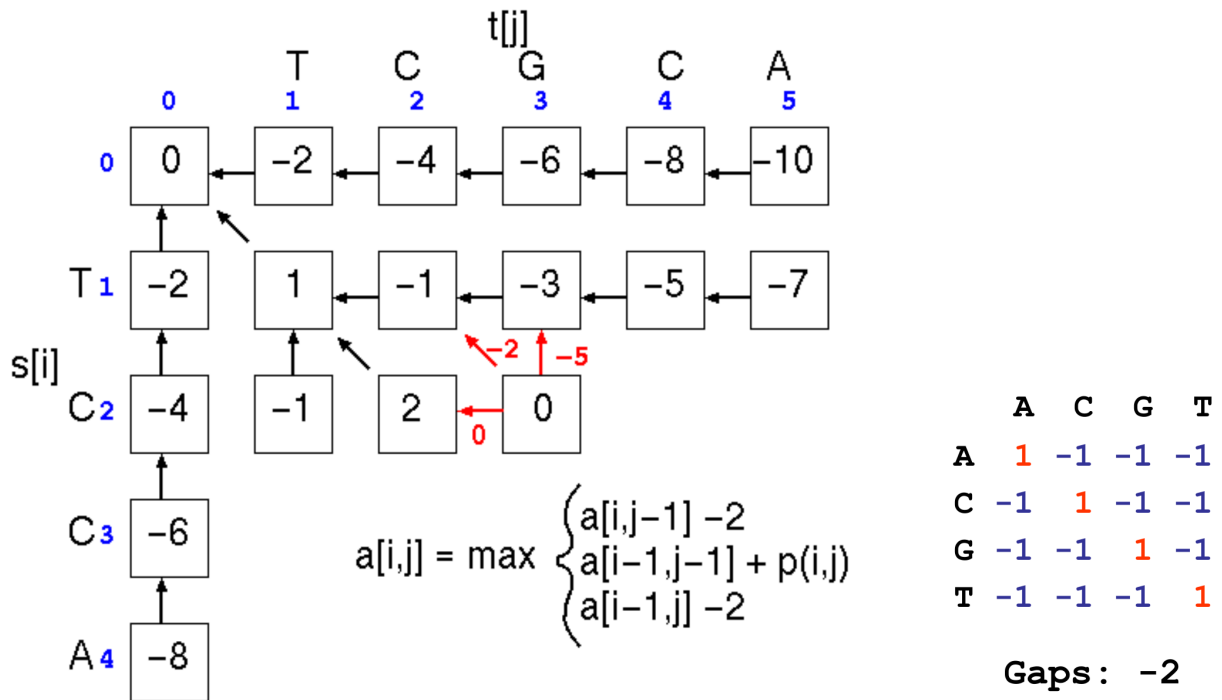
- **0** es el puntaje de la celda inicial (0, 0)
- Hay **T** en las ambas secuencias. Es un **match**: +1

Resultado

eje j: T
eje i: T

Para decidir qué valor ubicamos en la celda simplemente optamos por el que nos dé el **mayor score**, en este caso 1, y se marca el movimiento que lo produjo: un movimiento diagonal.

De esta manera podemos seguir completando la matriz,

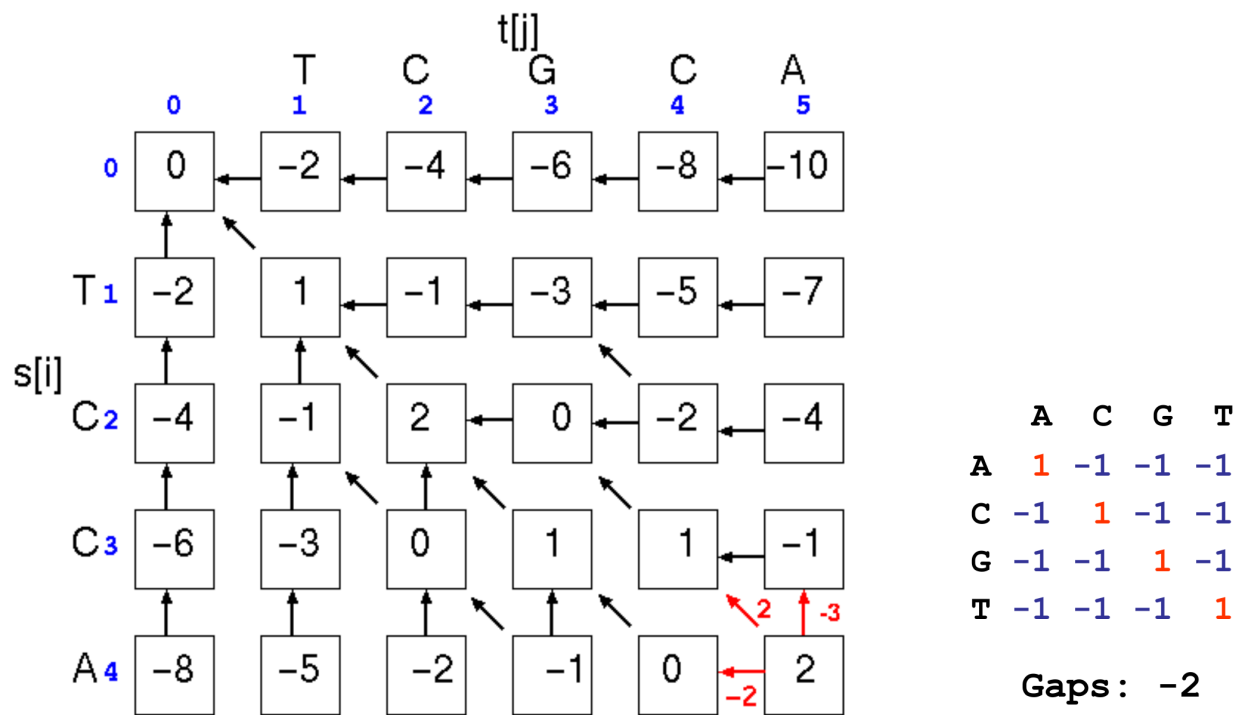


Observando la última celda computada, podemos ver que hay nuevamente 3 maneras de llegar a la misma,

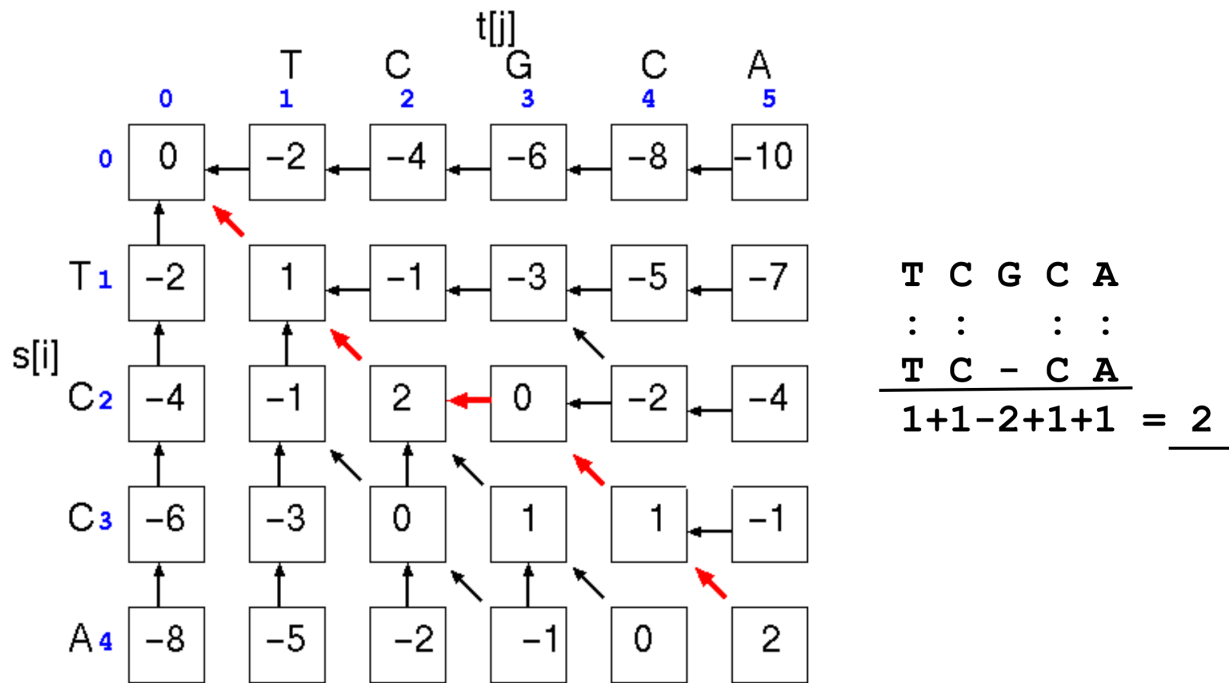
- Hacer un movimiento **vertical**, de (1, 3) a (2, 3): Es decir, introducir un **gap** en la secuencia horizontal j. Si (1, 3) tiene un *score* de -3, el nuevo *score* es: $-3 + \text{gap penalty} = -3 + (-2) = -5$ (flecha en dirección vertical).
- Hacer un movimiento **horizontal**, de (2, 2) a (2, 3). Es decir, introducir un gap en la secuencia vertical i. Si (2, 2) tiene un *score* de 2, el nuevo *score* es: $2 + \text{gap penalty} = 2 + (-2) = 0$ (flecha en dirección horizontal).
- Hacer un movimiento **diagonal**, de (1, 2) a (2, 3). Es decir, alinear los nucleótidos G y C. Si (1, 2) tiene un *score* de -1, el nuevo *score* es: $-1 + \text{mismatch} = -1 + (-1) = -2$ (flecha en dirección diagonal).

El máximo de los 3 scores calculados es: $\max(-5, 0, -2) = 0$, que corresponde al puntaje del **movimiento horizontal**. Entonces colocamos 0 en la celda (2, 3) y una flecha horizontal que indique el movimiento de (2, 2) a (2, 3).

Al completar todas las celdas de la matriz, podemos saber cuál es el puntaje de la celda ubicada en extremo inferior derecho, que en este caso resultó ser +2. Este también es el puntaje final del alineamiento.



Para reconstruir el mismo, se parte de la celda ubicada en extremo inferior derecho y se siguen las flechas hasta llegar a la celda de inicio, en el extremo superior izquierdo.



Las flechas en **rojo** resaltan el *path* del alineamiento,

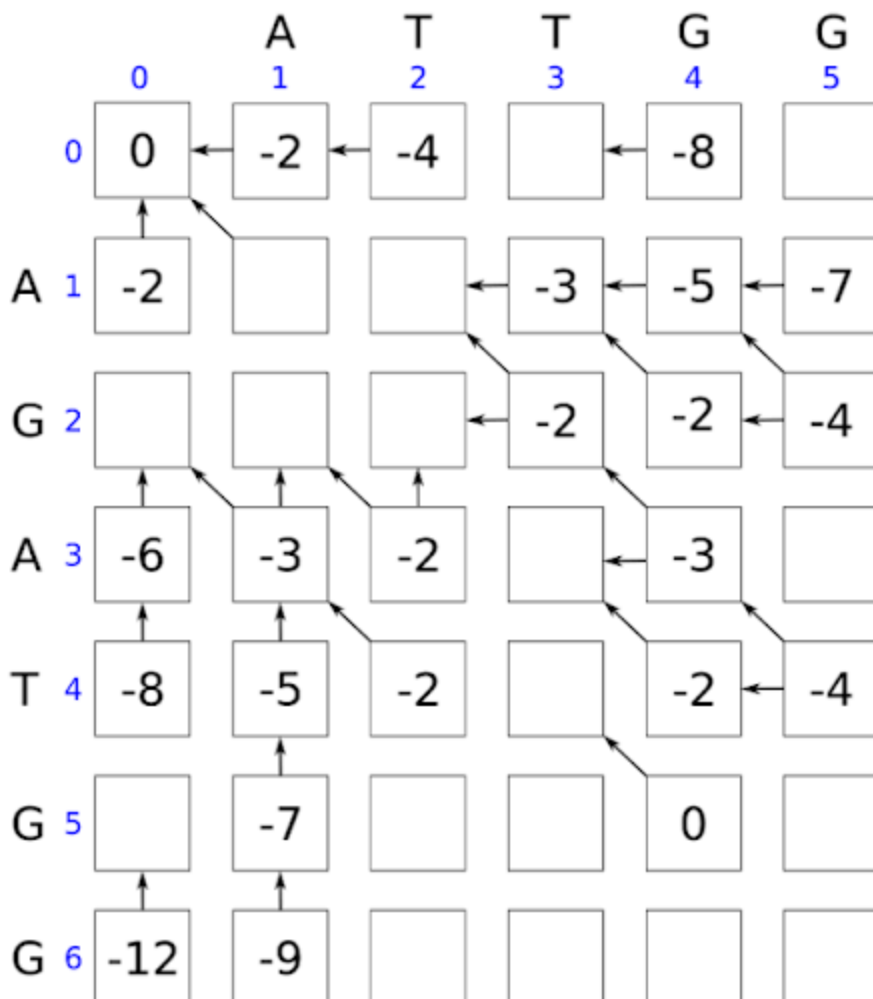
```
eje j: T C G C A
eje i: T C - C A
```

que podemos corroborar que es idéntico al *path* **1** del ejemplo que se planteó inicialmente.

Ejercicio 1

⇒ Paso 1

Realizá el alineamiento de las secuencias **ATTGG** con **AGATGG**, usando el esquema de puntajes: $M=1$, $m=-1$, $g=-2$.



⇒ Paso 2

Cuando termines el ejercicio anterior podés corroborar la solución que hallaste ingresando en [UniFreiburg-FreiburgRNATools](#).

Seguí las siguientes instrucciones para usar este recurso web:

a. Ingresá las dos secuencias que querés alinear en los recuadros de **Input Sequence a** y **Sequence b**.

Recordá que la secuencia que figura en tu matriz en sentido horizontal debe ser ingresada como **Sequence b** y la que figura en sentido vertical debe ser ingresada como **Sequence a**.

b. Seleccioná optimización de Similarity.

c. Completá los valores de tu esquema de *scoring*.

En el output podrás apreciar dos salidas:

- A la izquierda, los valores de la matriz de alineamiento. Si cliqueas sobre los valores de la matriz, vas a observar que el valor sobre el que te paraste se colorea en verde, mientras que las celdas que dieron origen a ese valor se colorean en rosa.
- A la derecha, se observa el alineamiento final, donde un match se esquematiza en con *, un mismatch con | y un gap con _.

⇒ Preguntas guía:

⇒ Pregunta 1

Reproducí el alineamiento que vimos como ejemplo al inicio (**TCGCA** con **TCCA**, esquema de puntajes: $M=1$, $m=-1$, $g=-2$) en la web de la [UniFreiburg-FreiburgRNATools](#)

¿Cuántas soluciones óptimas hay para este alineamiento? ¿Sucedo lo mismo para el alineamiento que realizaste en el ⇒ Paso 1? ¿Por qué?

⇒ Pregunta 2

Observá con detenimiento el output del panel de la izquierda (la matriz)

Seleccioná una celda. ¿Qué sucede cuando cliqueás en una celda y se colorea en **verde** y la celda aledaña a la misma en **rosa**?

¿A qué corresponde este coloreado o resaltado de las celdas?

Observá nuevamente la matriz del alineamiento que obtuviste en **1.2**. Cliqueá en la celda con puntaje -2 en la posición (A3, T2). Observá que se colorea en **verde** y **dos** celdas aledañas a la misma en **rosa**.

¿Entendés qué significa esto? ¿Podés relacionarlo con los dos caminos óptimos posibles que existen para este alineamiento?

Algoritmo de Smith-Waterman

El algoritmo de Smith-Waterman es un método de programación dinámica para el alineamiento local de secuencias, desarrollado por Temple Smith y Michael Waterman en 1981. A diferencia de Needleman-Wunsch, este algoritmo identifica las regiones de mayor similitud entre dos secuencias, sin necesidad de alinearlas en su totalidad.

¿Cómo funciona?

El algoritmo sigue una lógica similar a Needleman-Wunsch pero con una diferencia clave:

1. **Inicialización:** Construye una matriz donde los bordes se llenan con ceros (no con penalizaciones acumulativas).
2. **Llenado de la matriz:** Para cada celda, calcula el máximo entre:
 - Cero (reinicio del alineamiento)
 - Coincidencia/sustitución: puntuación diagonal + match/mismatch
 - Inserción: puntuación de arriba + penalización por gap
 - Deleción: puntuación de la izquierda + penalización por gap

3. **Trazado inverso (backtracking):** Comienza desde la celda con la puntuación máxima en toda la matriz y retrocede hasta encontrar un cero, reconstruyendo así el segmento de mayor similitud.

Diferencia clave con Needleman-Wunsch

Aspecto	Needleman-Wunsch	Smith-Waterman
Tipo de alineamiento	Global (toda la secuencia)	Local (solo regiones similares)
Valores en bordes	Penalizaciones acumulativas	Ceros
Puntuaciones negativas	Permiten penalizaciones acumulativas	Se convierten a cero (reinicio)
Punto de inicio del backtracking	Esquina inferior derecha	Celda con valor máximo
Mejor para...	Secuencias relacionadas en su totalidad	Secuencias con dominios conservados

Dot Plots

Un **Dot Plot** (o gráfico de puntos) es una herramienta visual para comparar dos secuencias biológicas (ADN, ARN o proteínas).

¿Cómo funciona?

- Se colocan **dos secuencias** en los ejes **X** e **Y**.
- Se dibuja un **punto (dot)** en cada coordenada donde los caracteres (bases o aminoácidos) coinciden y tienen un puntaje mayor al Criterio

de stringencia.

- Las **diagonales** en el gráfico indican regiones de similitud o alineamiento entre las secuencias.

Parámetros principales:

Parámetro	¿Qué hace?
Ventana (window)	Número de caracteres consecutivos que se comparan a la vez.
Criterio de stringencia (stringency)	Número mínimo de coincidencias dentro de la ventana para dibujar un punto.

Ejercicio 2

Vamos a comparar **dos secuencias de hemoglobina** usando la herramienta online [Dotlet](#).

Paso 1:

Dentro del superior de secuencias pegar las siguientes secuencias:

Secuencia 1:

MVLSPADKTNVKAAWGKVGAHAGEYGAEALERMFLSFPTTKTYFPHFDLSHGSAQVKGHGKK
VADALTNAVAHVDDMPNALSALSSDLHAHKLRVDPVNFK

Secuencia 2:

MVLSPADKTNVKAAWGKVGAHAGEYGAEALERMFLSFPTTKTYFPHFDLSHGSAQVKGHGKK
VADALTNAVAHVDDMPNALSALSSDLHAHKLRVDPVNFK

Qué hacer:

- Configura **Window = 10** .
- Hacé clic en "**Draw**".

Qué observar: Diagonal principal perfecta (100% identidad). No hay diagonales secundarias.

La matriz BLOSUM (BLOcks SUBstitution Matrix)

Es el estándar de facto para alinear proteínas. Fue creada por Henikoff y Henikoff en 1992.

Cómo funciona:

- Tiene un valor **específico para cada par de aminoácidos**
- No es binaria: cada una de las 190 sustituciones posibles tiene su propia puntuación
- Los valores se basan en la **probabilidad evolutiva** de que ocurra cada cambio

¿Cómo se calculan los valores?

- Se analizan bloques de alineamientos de proteínas reales
- Se comparan dos frecuencias:
 - i. **Frecuencia observada:** con qué frecuencia ocurre esa sustitución en la naturaleza
 - ii. **Frecuencia esperada al azar:** con qué frecuencia ocurriría si los aminoácidos se alinearan al azar
- El resultado se expresa como un **log-odds score**: $\log_2$ (observado / esperado)

Paso 2:

Dentro del superior de secuencias pegar las siguientes secuencias:

Secuencia 1:

MVLSPADKTNVKAAWGKVG AHAGEYGAEALERMFLSFPTTKTYFPHFDLSHGSAQVKGHGKK
VADALTNAVAHVDDMPNALSALSDLHAHKLRVDPVNFK

Secuencia 2:

MVLSPADKTNMVLSPADKTNGAHAGEYGAEALERMFLSFPTTKTYFPHFDLSHGSAQVKGHG

KKVADALTNAVAHVDDMPNALSALSDDLHAHKLRVDPVNFK

Explicación: La Secuencia 2 tiene el motivo "MVLSPADKTN" repetido al principio (20 aa), mientras que la Secuencia 1 lo tiene solo una vez. El resto de la secuencia (80 aa) es idéntico.

✎ Paso 3:

Dentro del superior de secuencias pegar las siguientes secuencias:

Secuencia 1:

MVLSPADKTNVKAAWGKVGAGHAGEYGAEALEEEEEEEEEEEEEEPHFDLSHGSAQVKGHGKK
VADALTNAVAHVDDMPNALSALSDDLHAHKLRVDPVNFK

Secuencia 2:

MVLSPADKTNVKAAWGKVGAGHAGEYGAEALEEEEEEEEEEEEEEPHFDLSHGSAQVKGHGKK
VADALTNAVAHVDDMPNALSALSDDLHAHKLRVDPVNFK

Explicación: Las Secuencias están compuestas por bloques de aminoácidos repetidos (E), lo que la hace de baja complejidad.

Qué observar: En la zona de baja complejidad, el dotplot se vuelve extremadamente denso, con múltiples diagonales paralelas y manchas de puntos que reflejan las repeticiones. La región de baja complejidad produce un patrón característico de "ruido" estructurado.

¿Cómo elegir los parámetros ideales?

No hay una combinación única y perfecta.

La elección de los parámetros (window y stringency) depende de varios factores:

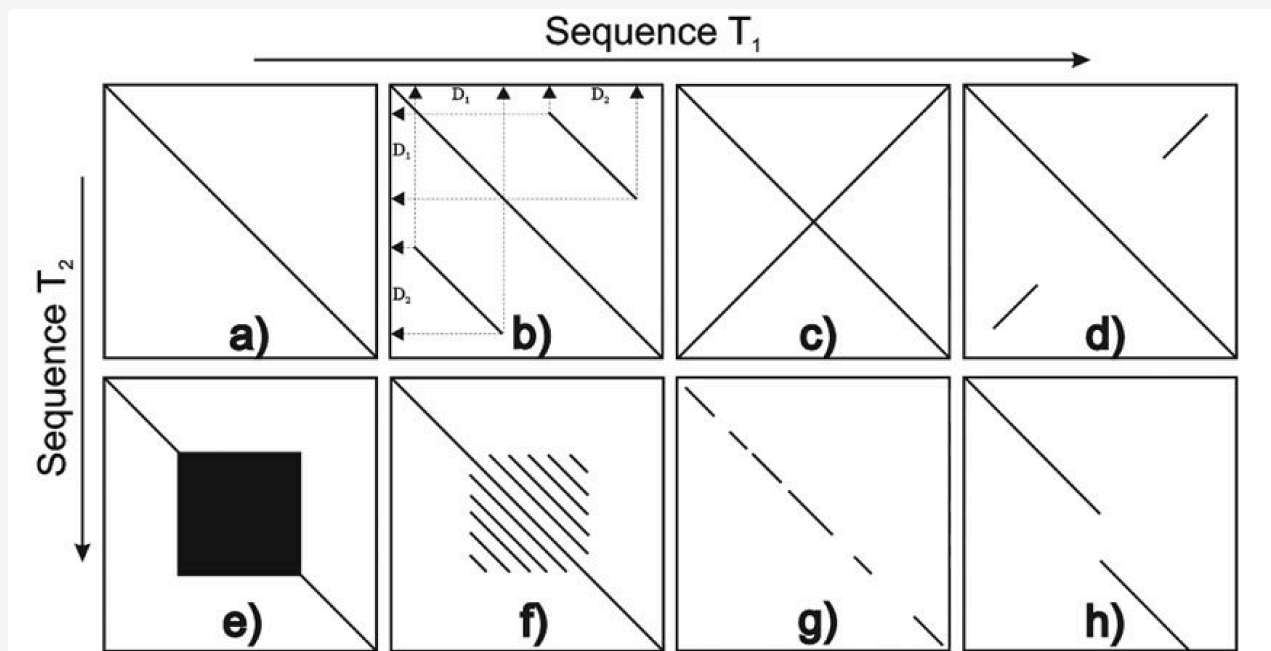
- **Tipo de secuencia:** ADN (4 letras) vs. Proteínas (20 aminoácidos). El ADN tiene más coincidencias aleatorias, por lo que necesita filtros más estrictos.
- **Similitud esperada:** Secuencias muy similares vs. secuencias

divergentes. Cuanto más divergentes sean, más ruido tendrás que filtrar.

- **Longitud de las secuencias:** Secuencias más largas generan más ruido estadístico.
- **Pregunta biológica:** ¿Buscas dominios conservados?
¿Repeticiones? ¿Reordenamientos?

La mejor estrategia es siempre experimentar con diferentes valores y observar cómo cambia el gráfico. No existe una combinación mágica; los parámetros óptimos son aquellos que mejor revelan la información biológica que te interesa visualizar.

Si aumentás estos parámetros podés ir eliminando fragmentos que corresponden a secciones compartidas más cortas, sin embargo existe una relación de compromiso, utilizar tamaño de ventana y umbral muy grandes nos llevan a perder información por lo que hay que seleccionarlos con cuidado. Aquí hay algunos patrones con los que te podés encontrar en este tipo de plots:



a) Match perfecto.

b) Repeticiones.

- c) Palíndromo.
- d) Repeticiones invertidas.
- e) Zonas de baja complejidad (microsatelites).
- f) Zonas altamente repetitivas (minisatelites).
- g) Secuencias con alta conservación.
- h) Inserción o delección.

Ejercicio 3

Ahora vamos a generar el mismo dot plot pero desde la **terminal** usando `dotmatcher` del paquete **Emboss**.

Requisitos:

- Tener Emboss instalado (`sudo apt install emboss` en Linux).
- Tener las secuencias en archivos `.fasta`.

🔗 Paso 1

Generá la estructura de directorios correspondiente y descargá la secuencia que vamos a comparar en esta sección ejecutando el siguiente comando:

```
wget "https://raw.githubusercontent.com/mercedesgarnham/Tutoriales/refs/heads/main/TP2/data/HS-ch1-fragment.fasta"
```

🔗 Paso 2

Visualizá las secuencias usando el comando `cat`

🔗 Paso 3

Generá un dotplot utilizando la secuencias descargadas en el paso 1

```
dotmatcher -graph pdf HS-ch1-fragment.fasta HS-ch1-fragment.fasta
```

Para limpiar el plot y quedarnos con los matches más significativos podemos jugar con dos parámetros:

- *windowSize*: Tamaño de ventana
- *threshold*: Umbral de ocurrencia

Esto quiere decir que `dotmatcher` sólo va a poner un punto cuando un fragmento del largo *windowSize* contenga un score mayor a *threshold*.

Por ejemplo:

```
dotmatcher -graph pdf -windowSize 50 -threshold 20 HS-ch1-fragment.fasta HS-ch1-fragment.fasta
```

Paso 4

Cambiá los parámetros *windowSize* y *threshold* hasta obtener un plot que te parezca adecuado. **¿Qué podés interpretar del mismo?** Identificá patrones.

Ejercicio 4

Hasta ahora trabajamos con los algoritmos de alineamiento de forma **casi manual**, entendiendo cómo funcionan por dentro los algoritmos.

Ahora vamos a dar un paso más: vamos a utilizar los algoritmos de alineamiento por **línea de comandos**.

Para eso vamos a usar **EMBOSS** (European Molecular Biology Open Software Suite), un paquete de software que diseña programas específicos para poder ejecutar desde la terminal y analizar secuencias biológicas de forma automatizada. Esto es lo que se usa en la práctica real cuando tenemos que procesar muchas secuencias o queremos integrar los análisis en flujos de trabajo (pipelines).

Consigna:

Determinar qué especies están más relacionadas utilizando la ribonucleasa pancreática de caballo (*Equus caballus*), ballena enana (*Balaenoptera acutorostrata*) y canguro rojo (*Macropus rufus*).



Equus caballus *Macropus rufus* *Balaenoptera acutorostrata*

🔖 Paso 1

Descargá los materiales de este TP ejecutando el siguiente comando:

```
wget "https://raw.githubusercontent.com/mercedesgarnham/Tutoriales/refs/heads/main/TP2/data/Balaenoptera_acutorostrata.fasta"

wget "https://raw.githubusercontent.com/mercedesgarnham/Tutoriales/refs/heads/main/TP2/data/Elephas_maximus.fasta"

wget "https://raw.githubusercontent.com/mercedesgarnham/Tutoriales/refs/heads/main/TP2/data/Equus_caballus.fasta"
```

🔖 Paso 2

Visualizá las secuencias usando el comando `cat`

🔖 Paso 3

Utilizá la herramienta de alineamiento global de EMBOSS `needle` (pueden leer el manual para ver que opciones admite ejecutando el comando `man needle`) para comparar las tres secuencias de a pares.

```
needle -gapopen 10 -gapextend 1 -asequence *secuencia_1* -bsequence
*secuencia_2* -outfile *salida*
```

🔖 Paso 4

Observá e interpretá las salidas obtenidas.

La salida contiene la siguiente información

Concepto	Definición
Identidad	Porcentaje de aminoácidos exactamente iguales
Similitud	Porcentaje de aminoácidos idénticos o similares (cambios conservados)
Gap	Hueco introducido para optimizar el alineamiento (indica inserción/delección)
Score	Puntuación total (matches - penalizaciones por gaps)
Alineamiento global	Alinea las secuencias de principio a fin (Needleman-Wunsch)

- ¿Qué secuencias son más similares?

Paso 5

Analizá árbol filogenético de la Fig. 1 del [paper](#) de O'Leary *et al.*, 2013. Sabiendo que los caballos y las ballenas pertenecen al clado *Euungulata* y los canguros al clado *Marsupialia*, ubicá estos clado en el árbol.

- ¿Esta información coincide con los resultados que obtuviste en el anterior?

Identidad, Similitud y Homología

Los términos identidad, similitud y homología se suelen utilizar como sinónimos por muchos investigadores, sin embargo no lo son.

- La **identidad** es una característica cuantitativa de un par de secuencias, donde se cuenta cuántos elementos (residuos, nucleótidos, aminoácidos etc) son idénticos entre ambas secuencias después de alinearlas.
- La **similitud** es una característica cuantitativa de un par de secuencias, donde se establece en qué grado estas se parecen (por

ejemplo aplicando los algoritmos antes vistos, utilizando un sistema de puntaje) después de alinearlas.

- La **homología**, por otro lado, es una característica cualitativa, dos secuencias SON o NO SON homólogas. Homología implica específicamente que el par de secuencias estudiadas *proviene de un mismo ancestro común*. Esta afirmación es completamente hipotética, ya que, salvo en contados casos, no se puede corroborar. Uno puede inferir que este es el caso dado la similitud observada en las secuencias actuales, sin tener acceso a las secuencias ancestrales.

Importante: Decir que un par de secuencias tiene N% de homología es TOTALMENTE incorrecto.

A partir de esta relación entre similitud y homología se pueden inferir relaciones entre diferentes especies, buscar posibles funciones de una secuencia desconocida, etc.

Cierre:

1. Que tipos de experimentos hicimos hoy?
2. Completá la tabla con los temas vistos hoy:

Tema	Algoritmo	Tipo de datos	Base de datos	Análisis del resultado
(completar)	(completar)	(completar)	(completar)	(completar)

3. Cumpliste con los objetivos del tutorial?

Objetivo	Se cumplió?
1. Entender el funcionamiento básico del algoritmo de alineamiento de pares de secuencias de Needleman-Wunsch.	Si / No
2. Aprender a interpretar un Dot-Plot, pudiendo identificar las regiones relevantes que contienen patrones.	Si / No
3. Comprender los conceptos de identidad, similitud y homología de secuencias, y establecer una clara diferencia entre los mismos.	Si / No